

Chapter 7: SET (One of the Most Important STL Containers)

If Vector is king of storage, then **Set is king of uniqueness**.

What Problem Does Set Solve?

Suppose:

```
1 5 2 1 5 3 2
```

You want:

```
1 2 3 5
```

- No duplicates
- Sorted automatically

Use:

```
set<int> s;
```

Properties of Set

1. Unique Elements

```
1 2 2 3 3 3
```

Stored as:

```
1 2 3
```

2. Sorted Automatically

Input:

```
5 1 8 2 4
```

Stored:

```
1 2 4 5 8
```

3. Implemented Using

Usually:

```
Red Black Tree
```

Balanced Binary Search Tree.

Header

```
#include <set>
```

Declaration

```
set<int> s;
```

Insert

insert()

```
s.insert(10);  
s.insert(5);  
s.insert(20);
```

Stored:

```
5 10 20
```

Duplicates ignored:

```
s.insert(10);
```

Still:

```
5 10 20
```

Complexity

```
O(log n)
```

Print Set

Using iterator:

```
for(auto it=s.begin();it!=s.end();it++)
{
    cout<<*it<<" ";
}
```

Output:

```
5 10 20
```

Range loop:

```
for(auto x:s)
{
```

```
    cout<<x<<" ";  
}
```

Size

```
cout<<s.size();
```

Output:

```
3
```

Empty

```
s.empty();
```

Returns:

```
true  
false
```

Find

Most important function.

find()

```
auto it=s.find(10);
```

Returns iterator.

If found:

```
it!=s.end()
```

Example:

```
if(s.find(10)!=s.end())  
{  
    cout<<"Found";  
}
```

Output:

Found

Complexity

$O(\log n)$

Count

Check existence.

count()

```
s.count(10);
```

Output:

1

Exists.

```
s.count(100);
```

Output:

```
0
```

Does not exist.

Since duplicates aren't allowed:

Output is only:

```
0 or 1
```

Complexity

```
O(log n)
```

Erase

By Value

```
s.erase(10);
```

Before:

```
5 10 20
```

After:

```
5 20
```

Complexity

```
O(log n)
```

By Iterator

```
auto it=s.find(20);  
  
s.erase(it);
```

Clear

```
s.clear();
```

Removes everything.

Swap

```
s1.swap(s2);
```

Lower Bound

Very important.

Definition:

First element

```
>= x
```

Example

```
set<int> s={10,20,30,40};
```

```
auto it=s.lower_bound(25);
```

Output:

```
30
```

Because:

```
30 >= 25
```

First such element.

Example:

```
s.lower_bound(20);
```

Output:

```
20
```

Complexity

```
 $O(\log n)$ 
```

Upper Bound

Very important.

Definition:

First element

```
> x
```

Example:

```
set<int> s={10,20,30,40};
```

```
auto it=s.upper_bound(20);
```

Output:

```
30
```

```
auto it=s.upper_bound(25);
```

Output:

```
30
```

Difference

Set:

```
10 20 30 40
```

For:

```
20
```

Lower Bound:

20

Upper Bound:

30

Reverse Traversal

```
for(auto it=s.rbegin();it!=s.rend();it++)
{
    cout<<*it<<" ";
}
```

Output:

40 30 20 10

Time Complexity Table

Operation	Complexity
insert	$O(\log n)$
erase	$O(\log n)$
find	$O(\log n)$
count	$O(\log n)$
lower_bound	$O(\log n)$
upper_bound	$O(\log n)$
size	$O(1)$

Vector vs Set

Feature	Vector	Set
Duplicates	Allowed	Not Allowed
Sorted	No	Yes
Search	$O(n)$	$O(\log n)$
Insert	$O(1)$ end	$O(\log n)$

Common Interview Uses

Remove Duplicates

```
vector<int> v={1,2,2,3,1};  
  
set<int> s(v.begin(),v.end());
```

Result:

```
1 2 3
```

Check Duplicate

```
if(s.count(x))
```

Sorted Unique Data

```
set<int>
```

Perfect.

Practice Questions

Easy

Q1

Insert:

```
10 5 20 10
```

Print set.

Output:

```
5 10 20
```

Q2

Check if 15 exists.

Q3

Remove 10.

Print set.

Q4

Print set in reverse.

Q5

Count distinct elements.

Input:

```
1 2 2 3 1 5
```

Output:

```
4
```

Medium

Q6

Remove duplicates from array.

Q7

Find first element $\geq x$.

Use:

```
lower_bound()
```

Q8

Find first element $> x$.

Use:

```
upper_bound()
```

Q9

Find smallest missing positive number.

Hint:

```
set
```

Q10

Print all unique elements sorted.

Interview Level

Q11

Contains Duplicate

LeetCode #217

Q12

Intersection of Two Arrays

LeetCode #349

Q13

Longest Consecutive Sequence

LeetCode #128

(Uses set/unordered_set)

Q14

Kth Smallest Element

Use ordered set logic.

Q15

Find predecessor and successor.

Use:

```
lower_bound()  
upper_bound()
```

Common Mistakes

Mistake 1

Trying:

```
s[0]
```

✗ Not allowed

Set has no indexing.

Mistake 2

Trying:

```
s.push_back(10);
```

✗ Wrong

Use:

```
s.insert(10);
```

Mistake 3

Dereferencing end()

```
*s.end()
```

✗ Undefined Behavior

Revision Sheet

Header

```
#include <set>
```

Declaration

```
set<int> s;
```

Insert

```
s.insert(x);
```

Remove

```
s.erase(x);
```

Search

```
s.find(x);  
s.count(x);
```

Bounds

```
s.lower_bound(x);  
s.upper_bound(x);
```

Utilities

```
s.size();  
s.empty();  
s.clear();  
s.swap();
```

Iterators

```
begin()  
end()  
rbegin()  
rend()
```

Complexity

```
insert      O(log n)  
erase      O(log n)  
find       O(log n)  
count      O(log n)  
lower_bound O(log n)  
upper_bound O(log n)
```

Mental Trigger

When you see:

- Unique elements
- Remove duplicates
- Sorted unique values
- Fast search + sorted order
- Predecessor / Successor

Think:

SET

Assignment

Solve:

1. Count distinct elements.
 2. Remove duplicates.
 3. Find predecessor.
 4. Find successor.
 5. Kth smallest element.
 6. Intersection of two arrays.
 7. Longest consecutive sequence.
 8. Contains duplicate.
-